

Notes on Embedding 3SAT into Graph Problems

1 Problem Statement: 3SAT

Definition 1 (3SAT). *An instance of 3SAT consists of Boolean variables $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$, where each clause is a disjunction of exactly three literals:*

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

and each literal ℓ is either x_i or $\neg x_i$ for some i . The decision question is whether there exists an assignment $\alpha : \{x_i\} \rightarrow \{T, F\}$ satisfying all clauses.

2 High-Level Reduction Template

Most $3SAT \rightarrow$ graph reductions follow the same blueprint:

Construct a graph instance $G(\varphi)$ from a 3CNF formula φ such that:

$$\varphi \text{ is satisfiable} \iff G(\varphi) \text{ has a feasible solution (for the target graph problem).}$$

Modular structure

1. **Variable gadgets:** enforce a binary choice (True vs False).
2. **Clause gadgets:** enforce “at least one literal is True”.
3. **Consistency wiring:** ensure each literal occurrence respects the variable choice.
4. **Soundness/completeness:** prove both directions of the iff statement.

3 Variable Gadgets

3.1 Purpose

A variable gadget for x_i must encode a *single global* truth choice, used consistently across all appearances of x_i and $\neg x_i$ in clauses.

3.2 Common patterns

(A) Two-choice path gadget (routing) Two internally disjoint branches correspond to $x_i = T$ and $x_i = F$. Any feasible global solution must pick exactly one branch (e.g., a path, a set of disjoint paths, a shortest path under constraints).

(B) Matching choice gadget (selection) A subgraph admits exactly two “modes” under a matching/perfect matching constraint; choosing one mode corresponds to $x_i = T$ and the other to $x_i = F$. This is common in reductions to matching/factor problems.

(C) Orientation/circulation gadget (directed constraints) In directed settings, orienting a cycle or selecting a circulation pattern can represent a truth value and constrain what edges can be used to satisfy clauses.

3.3 Design checklist

A correct variable gadget should satisfy:

- **Exclusivity:** cannot simultaneously represent T and F.
- **Extractability:** from any feasible solution, we can read off T vs F.
- **Interface:** literal ports connect cleanly to clause gadgets.

4 Clause Gadgets

4.1 Purpose

Each clause $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$ must be enforced by the graph so that *feasibility* requires at least one of its literals to be “activated” (i.e., set to True by the variable gadget).

4.2 Standard approaches

(A) OR-gadget via availability Clause gadget is feasible if at least one literal-connection edge/path is available; if all three are blocked (all false), the gadget becomes infeasible.

(B) Flow-based OR-gadget Require one unit of flow to enter the clause gadget. Flow can enter only through a port corresponding to a True literal; thus feasibility implies at least one literal is True.

(C) Obstruction gadget If all literals are false, the global solution is forced to contain an impossible structure (e.g., must traverse a forbidden edge/cut, violate planarity constraints, exceed a budget).

4.3 Design checklist

- **Local correctness:** clause gadget enforces “ ≥ 1 literal true”.
- **No loopholes:** no alternate route that satisfies the gadget without a true literal.
- **Constant size:** typically $O(1)$ vertices/edges per clause.

5 Consistency: Wiring Literals to Variables

5.1 Literal ports

For each occurrence of a literal ℓ in clause C_j , create a connection (edge/path/interface) from:

- the **True side** of variable gadget x_i if $\ell = x_i$, or
- the **False side** of variable gadget x_i if $\ell = \neg x_i$,

into the corresponding port of the clause gadget for C_j .

5.2 Preventing inconsistent assignments

Common enforcement methods (depending on the target graph problem):

- capacity constraints (a port usable only if variable mode allows it),
- disjointness constraints (cannot “mix” truth modes without conflict),
- directed edges and reachability constraints,
- shared resources/budgets that make cheating solutions impossible.

6 Polynomial Size Bound

To be a valid polynomial-time reduction, the constructed graph must be polynomial in $n + m$. Typical guarantee:

$$|V(G)|, |E(G)| = O\left(n + m + \sum_i \deg(x_i)\right) = O(n + m).$$

This holds if each gadget is constant size and each literal occurrence contributes $O(1)$ wiring.

7 NP-Hardness Proof Structure

A clean write-up always includes two directions.

7.1 Completeness

Claim 1 (Completeness). *If φ is satisfiable, then $G(\varphi)$ has a feasible solution (for the target graph problem).*

Proof sketch. Given a satisfying assignment, set each variable gadget to the corresponding mode (T/F). For each clause, at least one literal is True, so the corresponding clause gadget has at least one active port, making it feasible. Combine the local choices into a global feasible solution. $\square$

7.2 Soundness

Claim 2 (Soundness). *If $G(\varphi)$ has a feasible solution, then φ is satisfiable.*

Proof sketch. From any feasible solution, read off each variable gadget’s mode (extract an assignment). Consistency wiring ensures that literal ports reflect this mode. Clause gadget feasibility implies at least one literal port is active per clause, hence each clause has a satisfied literal. Therefore the extracted assignment satisfies φ . $\square$

8 Membership in NP (for NP-Completeness)

If the goal is NP-completeness, we also show that the graph problem is in NP.

Typical certificate

Depending on the graph problem, a certificate could be:

- an explicit path / set of paths,
- a matching / factor set,
- a flow assignment,
- an ordering / labeling / orientation.

Verification

Verification is usually polynomial-time: check local constraints, capacities, adjacency, disjointness, and objective bounds.

9 Canonical Examples

Many classic NP-hardness results use 3SAT embeddings:

- Hamiltonian Path/Cycle (route must “choose” literals),
- Vertex Cover / Independent Set (selection encodes truth),
- Graph Coloring (colors encode assignments),
- Disjoint Paths (routing encodes choices),
- Flow/Circulation problems (literal availability controls feasibility).